

Change we need

Kshitij Sobti

kshitij.sobti@thinkdigit.com

Change is the only thing that is constant, yet it is the thing for which we are least prepared. In some ways we measure progress by how many changes we make in completing something.

Human error is unavoidable, and in today's computing world, we have become used to undoing and redoing our changes. We have now come to rely on such measures to the extent that otherwise risky steps becomes trivial. Yet the scope of such systems is often limited to the application we use. There is no easy cure for a file deleted weeks ago, correcting a mistake you have been building upon for a long time is no easy task.

Such problems are common in the programming world, where a large number of people modify a single code base towards a single common goal. How can changes be tracked? How can we ensure two people don't change the same lines of code simultaneously? There are numerous methods to track and maintain changes made to the source code. Let us see how you can use these technologies for purposes other than coding.

The need for revision control

If you've ever worked on a complicated project, you will surely understand the benefits of saving multiple copies of your work in progress. Imagine working on your history project for weeks, only to realise that the original introduction would fit in better. Without backups at multiple stages – you're helpless.

Often we accomplish this by repeatedly saving the file with a different name, hopefully with some kind of pattern such as "HistoryProject-v1.doc", "HistoryProject-v2.doc" or "HistoryProject-20090615.doc". However, over time this will surely create a mess.

The problems complicate even further when you have multiple people working on the same project. How do you reconcile the barrage of changes that a team can inflict?

It may seem simple to have the file hosted on a shared server where everyone can access and modify it. However, if everyone were to save files affixed with their name and timestamps, you would soon end up with an unmanageable mess

of files of which only a few will ever be of any use, and many changes will be lost as people will use outdated revisions of the file and build on those.

Wouldn't it be great if your file system itself could store each change separately? Such that it would be invisible until needed? Many such file systems indeed exist (ext3cow, wayback, tux3 etc). However a simpler solution, which doesn't involve shifting to *NIX and messing around with kernel modules is to use revision control software.

We can categorise revision control systems based on how the system manages its repository of data, and how it manages to avoid conflicts.

Two approaches to resolving conflicts as multiple people try to modify the same file at the same time are the lock-modify-unlock model, and the copy-modify-merge model.

Lock-Modify-Unlock

This system is remarkably simple. Whenever one member of a team needs to modify a particular file, it is locked, so that no one else can make changes to it until it is unlocked again.

Think of it as setting a file as read only for everyone else while you modify it to your pleasure; once you are done, you can "unlock" the file again and other people can begin their work.

The disadvantages here are quite evident. Firstly, you are at the mercy of the first person who locked the file, even though in most cases the changes you might make would not conflict with

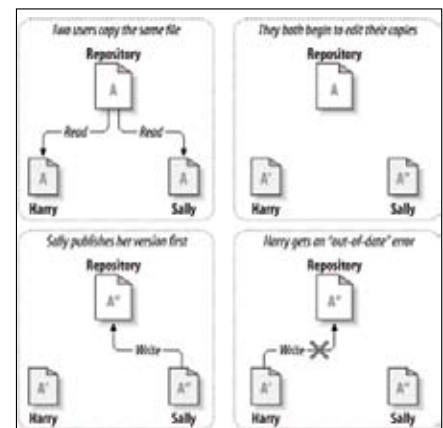
what they are doing. This is clearly a loss of productivity, especially if that file is essential to your work.

Then again, if a person locks a file and forgets to unlock before going on a vacation / quitting his job / or even going to sleep, you are at a deadlock until an administrator frees the shackles.

Finally, it doesn't even really solve the problem completely since changes in one file may still conflict with the changes being made in another.

Copy-Modify-Merge

A more elegant solution for managing simultaneous access, is to simply give it. Each user can create their own copy of the

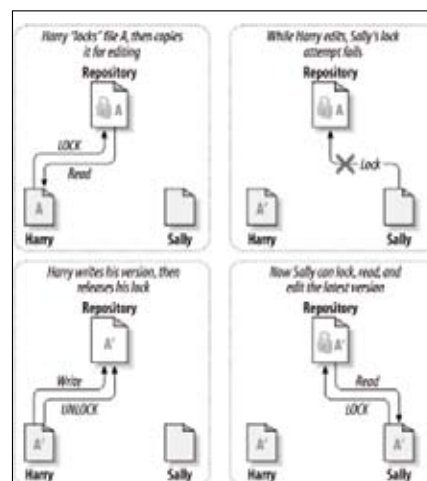


In the Modify-Merge scheme, users independently modify the documents

file which they then proceed to modify as if no one is watching.

Finally, they can "merge" back their changes. By this, we mean that the revision system itself determines if there is any conflict in the changes made in the file contents. In most cases, different people will modify different parts of the file and the system will work quite well. However, in the rare case of a conflict that cannot be solved by the software itself, users can be informed of the problem and can make the decision themselves.

To see how this would work, let us take revision 1 of a document, which two people AlphaGeek and BetaDood copy simultaneously. AlphaGeek, alpha as he is, manages to complete his changes first and merges the changes into the main document bringing it up to par with his copy.



An example of Lock-Modify-Unlock